Day 134 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 20, Day 2 reference

Watch: Ian Goodfellow: Generative Adversarial Networks (NIPS 2016 tutorial) | Generative Adversarial Networks (GANs) - Explained (DataMListic)

`day134_gan_from_scratch.py`, segment by segment, in the order `main()` uses things. The GAN parts (AdamGAN, softplus, d_loss, g_loss, generate, g_grads, gan_step) and the 1-D data set are new; the networks are Day 126's init_net, forward, backward and Adam, the judge is Day 116's fit_softmax, and nearest-image distances use Day 119's sqdist, all imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: G = the generator · D = the discriminator · z, Z = noise · F = fakes · lr, lf = D's logits on real and fake · D(x) = sigmoid(logit) = P(real)

Segment 0 — Setup

Imports

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
for w in ("week-19-unsupervised-learning-ii", "week-18-unsupervised-learning-i",
    "week-17-classical-ml-kernels-ensembles"):
    sys.path.insert(0, os.path.join(HERE, "..", w))
with contextlib.redirect_stdout(io.StringIO()):
    import day126_exercise as d126
    import day126_autoencoders as d126m
    import day119_exercise as d119
    import day116_exercise as d116
```

- `for w in (...): sys.path.insert(...)` — three earlier weeks' folders, so their exercise files and Day 126's loader can be imported.
- `with contextlib.redirect_stdout(io.StringIO()):` — the imported lessons print their own checks; this hides that output.

Segment 1 — The two players

AdamGAN: Day 126's Adam with a momentum setting

MATH

$$m \leftarrow b_1\, m + (1 - b_1)\, g, \quad v \leftarrow 0.999\, v + 0.001\, g^2, \quad w \leftarrow w - lr\, (m / (1 - b_1^{\,t})) / (\sqrt{(v / (1 - 0.999^t))} + 10^{-8})$$

CODE

```
class AdamGAN(d126.Adam):
    def __init__(self, net, lr=2e-4, b1=0.5):
        super().__init__(net, lr); self.b1 = b1
    def step(self, net, grads):
        self.t += 1; c1, c2 = 1 - self.b1 ** self.t, 1 - 0.999 ** self.t
        for (W, b), (gW, gb), m, v in zip(net, grads, self.m, self.v):
            for p, g, mm, vv in ((W, gW, m[0], v[0]), (b, gb, m[1], v[1])):
                mm *= self.b1; mm += (1 - self.b1) * g; vv *= 0.999; vv += 0.001 * g * g
                p -= self.lr * (mm / c1) / (np.sqrt(vv / c2) + 1e-8)
```

- `class AdamGAN(d126.Adam):` — inheritance: AdamGAN is an Adam, and only what is written here changes.
- `super().__init__(net, lr)` — run Day 126's constructor first (it makes the m and v buffers), then add b1.
- `mm *= self.b1; mm += (1 - self.b1) * g` — the running average of gradients, in place; b1 = 0.5 forgets old gradients faster than 0.9.

softplus, d_loss, g_loss

MATH

$$L_D = -\text{mean log } D(x) - \text{mean log}(1 - D(G(z))), \quad L_G = -\text{mean log } D(G(z)) \;\; \text{(non-saturating)}$$

CODE

```
def softplus(Z):
    return np.maximum(Z, 0) + np.log1p(np.exp(-np.abs(Z)))
def d_loss(lr, lf):
    loss = softplus(-lr).mean() + softplus(lf).mean()
    return float(loss), (d126.sigmoid(lr) - 1) / len(lr), d126.sigmoid(lf) / len(lf)
def g_loss(lf, kind="non-saturating"):
    if kind == "non-saturating":
        return float(softplus(-lf).mean()), (d126.sigmoid(lf) - 1) / len(lf)
    return float(-softplus(lf).mean()), -d126.sigmoid(lf) / len(lf)
```

- `np.maximum(Z, 0) + np.log1p(np.exp(-np.abs(Z)))` — $\log(1 + e^z)$ written so exp never sees a large positive number.
- `softplus(-lr).mean() + softplus(lf).mean()` — –log sigmoid(l) = softplus(–l) and –log(1 – sigmoid(l)) = softplus(l): the losses straight from logits.
- `(d126.sigmoid(lr) - 1) / len(lr)` — the derivative of softplus(–l) is sigmoid(l) – 1, divided by the batch size because of the mean.
- `if kind == "non-saturating":` — the default string argument picks which generator loss is used.

generate and g_grads: the gradient through the discriminator

MATH

$$\partial L_G/\partial \theta_G = (\partial L_G/\partial D\text{-logit})\,(\partial D\text{-logit}/\partial x)\,(\partial x/\partial \theta_G), \quad x = G(z)$$

CODE

```
def generate(G, Z, out="sigmoid"):
    A = d126.forward(G, Z, ["relu"] * (len(G) - 1) + ["linear"])
    return (d126.sigmoid(A[-1]) if out == "sigmoid" else A[-1]), A
def g_grads(G, D, Z, out="sigmoid", kind="non-saturating"):
    dacts = ["relu"] * (len(D) - 1) + ["linear"]; gacts = ["relu"] * (len(G) - 1) + ["linear"]
    F, AG = generate(G, Z, out); Af = d126.forward(D, F, dacts); lg, gf = g_loss(Af[-1], kind)
    _, dF = d126.backward(D, Af, dacts, gf)
    dL = dF * F * (1 - F) if out == "sigmoid" else dF
    return lg, d126.backward(G, AG, gacts, dL)[0], Af
```

- `["relu"] * (len(G) - 1) + ["linear"]` — a list of activations: ReLU for every hidden layer, linear for the last.
- `(d126.sigmoid(A[-1]) if out == "sigmoid" else A[-1]), A` — a conditional expression inside a tuple: pixels squeezed to [0, 1], or raw values.
- `_, dF = d126.backward(D, Af, dacts, gf)` — Day 126's backward returns (weight gradients, gradient at the input); the underscore discards D's weight gradients.
- `dF * F * (1 - F)` — through the sigmoid: its derivative is F (1 – F).
- `d126.backward(G, AG, gacts, dL)[0]` — then back through the generator itself; [0] keeps only the weight gradients.

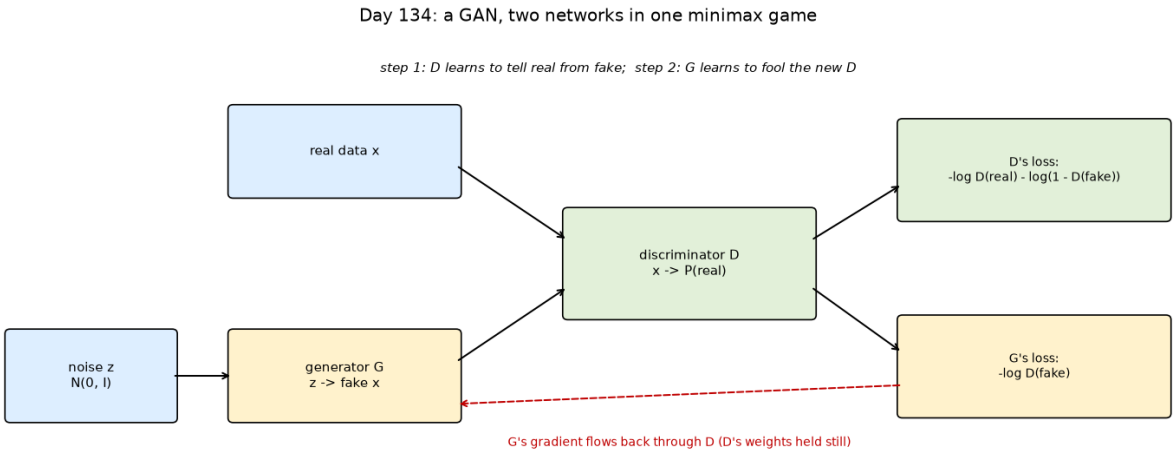
gan_step: one round of the game

CODE

```
def gan_step(G, D, oG, oD, X, rng, k, out="sigmoid", kind="non-saturating", d_steps=1):
    dacts = ["relu"] * (len(D) - 1) + ["linear"]; n = len(X)
    for _ in range(d_steps):
        F, _ = generate(G, rng.standard_normal((n, k)).astype(np.float32), out)
        Ar, Af = d126.forward(D, X, dacts), d126.forward(D, F, dacts)
        ld, gr, gf = d_loss(Ar[-1], Af[-1])
        g1, _ = d126.backward(D, Ar, dacts, gr); g2, _ = d126.backward(D, Af, dacts, gf)
        oD.step(D, [(a[0] + b[0], a[1] + b[1]) for a, b in zip(g1, g2)])
    lg, grads, Af = g_grads(G, D, rng.standard_normal((n, k)).astype(np.float32), out, kind)
    oG.step(G, grads)
    return ld, lg, float(d126.sigmoid(Ar[-1]).mean()), float(d126.sigmoid(Af[-1]).mean())
```

- `for _ in range(d_steps):` — the discriminator can take several steps per generator step.
- `rng.standard_normal((n, k)).astype(np.float32)` — a fresh batch of noise, as float32 like the networks.
- `[(a[0] + b[0], a[1] + b[1]) for a, b in zip(g1, g2)]` — a list comprehension adding the real-half and fake-half gradients, layer by layer.
- `oG.step(G, grads)` — only the generator's weights change in step 2.

SYSTEM DIAGRAM



The two players, and where each gradient goes.

Segment 2 — The experiments

w1 and the 1-D game

MATH

$$W_1(P, Q) = mean_i\, |a_{(i)} - b_{(i)}| \quad \textit{for two sorted samples of equal size}$$

CODE

```
def w1(A, B):
    return float(np.mean(np.abs(np.sort(A.ravel()) - np.sort(B.ravel()))))
```

- `np.sort(A.ravel())` — flatten to one dimension, then sort: in 1-D the best pairing of two samples matches them in sorted order.
- `np.mean(np.abs(...))` — the average distance each sample must move: the 1-D Wasserstein distance.

demo_optimal_d: the value of the game, by numerical integration

MATH

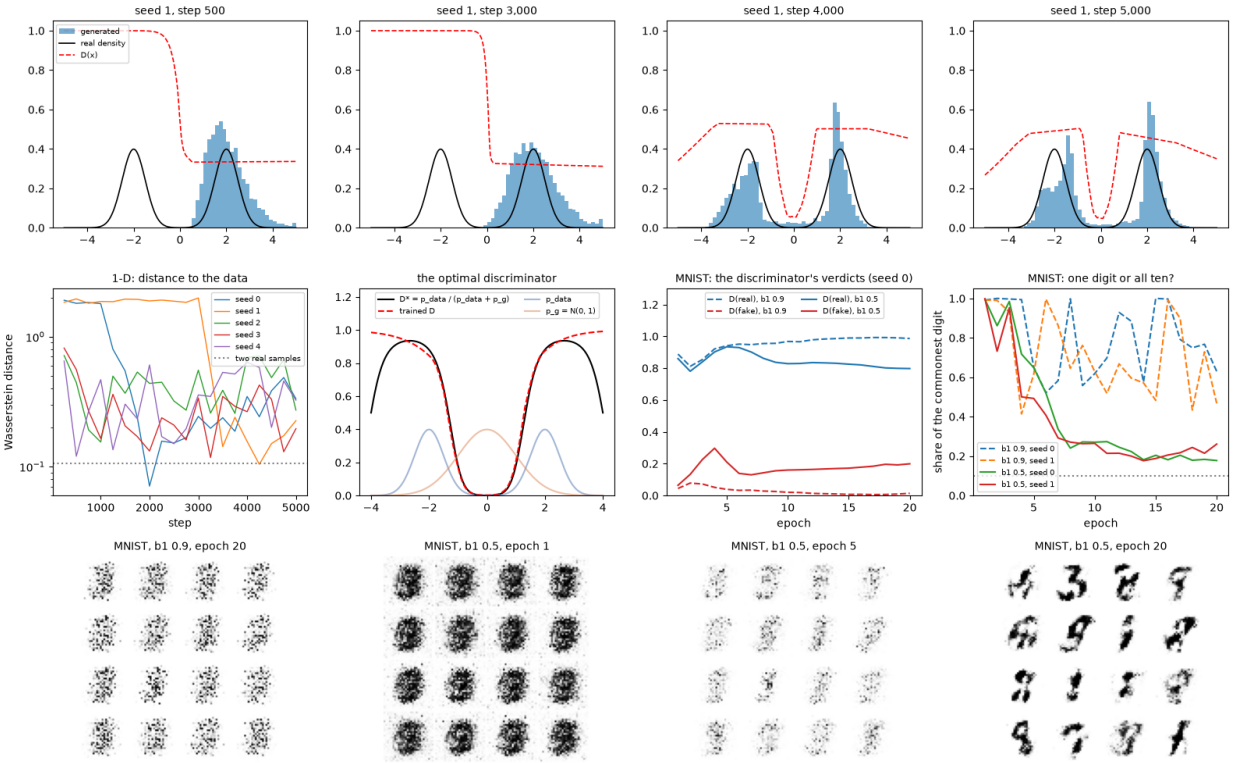
$$V(D^*) = \int p_d \log(p_d / (p_d + p_g)) + p_g \log(p_g / (p_d + p_g))\, dx = -\log 4 + 2\, JSD(p_d, p_g)$$

CODE

```
xs = np.linspace(-12, 12, 200001); dx = xs[1] - xs[0]; pd, pg = p_real1d(xs), p_g(xs); m = 0.5 *
    (pd + pg)
kl = lambda p, q: float(np.sum(p * np.log((p + 1e-300) / (q + 1e-300))) * dx)
jsd = 0.5 * kl(pd, m) + 0.5 * kl(pg, m)
v_star = float(np.sum(pd * np.log(pd / (pd + pg) + 1e-300) + pg * np.log(pg / (pd + pg) + 1e-300))
    * dx)
```

- `np.sum(f(xs)) * dx` — an integral as a sum of thin strips: fine enough here because the grid has 200,001 points.
- `kl = lambda p, q: ...` — a one-line function: the Kullback–Leibler divergence of two densities on the grid.
- `+ 1e-300` — keeps log away from log(0) far out in the tails, where both densities underflow to zero.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `class B(A): ... super().__init__(...)` — inherit a class and extend its constructor.
- `np.log1p(x)` — log(1 + x), accurate when x is tiny.
- `_, dF = f(...)` — unpack a pair and ignore the first item.
- `np.ndindex(W.shape)` — every index tuple of an array, for looping over all its entries.
- `lambda p, q: ...` — a small unnamed function.
- `np.sort(a.ravel())` — a flattened, sorted copy.

End of Day 134 syntax reference. For the experiments and what they showed, see the Day 134 complete guide.